Method of Procedure (MoP) - AI-Assisted Per-Cell Monthly Traffic Forecasting using FPM

We will take KPI data that looks exactly like what our network generates today, use this historical monthly per-cell traffic KPIs to automatically forecast traffic for the next 6 months, using a ML model that understands time trends

Input: Sample KPI report - 30 months.csv – 100 cells for 30 months
Primary Forecasting KPIs: Traffic_GB
Models: FPM - Facebook Prophet Model (one of the best models for time- series data)
Forecast horizon: 6 months

Python Libraries required: to be installed in anaconda prompt

- pandas
- numpy
- prophet
- matplotlib

pip install pandas prophet matplotlib numpy

Tools used: Juypter notebook (anaconda) and Streamlite app

High-Level Flow (Explain Before Coding)

1. Load real KPI data
2. Prepare AI-ready features
3. Train the FPM model
4. Validate accuracy
5. Forecast next 6 months

Step 1: Upload Sample KPI data generated from the LTE generator

```
import pandas as pd
from prophet import Prophet
import numpy as np

df = pd.read_csv("Sample KPI report - 30 months.csv")

print("Sample KPI file loaded successfully")
print("Rows:", len(df))
print("Columns:", df.columns.tolist())

df.head()
```

STEP 2 – Data Transformation & Validation

```
# Convert & align Date column and standardise all dates to month-end
# Convert Date column (explicit day-first format)
```

```
df["Date"] = pd.to_datetime(
    df["Date"],
    dayfirst=True,
    errors="coerce"
)

df = df.dropna(subset=["Date"])

print("After fixing dates:", len(df))

# Remove duplicate Cell–Date records
df = df.sort_values(["Cell", "Date"])
df = df.drop_duplicates(subset=["Cell", "Date"], keep="first")

print("After duplicate Cell-Date removal:", len(df))

# Remove rows with missing values

df = df.dropna(subset=["Date", "Cell", "Traffic_GB"])
print("After removing missing values:", len(df))

# Validate KPI value ranges (based on your local rules - telecom sanity)

df = df[df["Traffic_GB"] > 0]
print("After removing non-positive traffic values:", len(df))

# Sort Data (critical for time series)
df = df.sort_values(["Cell", "Date"])

# Final data quality summary

print("\nFINAL DATA QUALITY SUMMARY")
print("----------------------------")
print("Total rows after validation:", len(df))
print("Number of unique cells:", df["Cell"].nunique())
print("Date range:", df["Date"].min(), "to", df["Date"].max())

df.describe()

# Save validated KPI dataset

validated_file = "Validated_KPI_Report_30_Months.csv"
df.to_csv(validated_file, index=False)

print(f"\nValidated KPI file saved as: {validated_file}")

# Reload validated KPI file for downstream steps
```

```
df = pd.read_csv(validated_file)
df["Date"] = pd.to_datetime(df["Date"])

print("Validated KPI file reloaded for AI training.")
print("Rows available for modeling:", len(df))
```

STEP 3 –
FPM used for Traffic Forecasting - Traffic is a time-based KPI. FPM is designed specifically for time-series forecasting

STEP 4 – Per-Cell Model Training & Forecasting

```
future_months = 6
forecast_rows = []

for cell, cell_df in df.groupby("Cell"):

    prophet_df = cell_df.rename(
        columns={"Date": "ds", "Traffic_GB": "y"}
    )[["ds", "y"]]

    model = Prophet(
        yearly_seasonality=True,
        weekly_seasonality=False,
        daily_seasonality=False
    )

    model.fit(prophet_df)

    future = model.make_future_dataframe(
        periods=future_months,
        freq="ME"
    )

    forecast = model.predict(future)
    future_forecast = forecast.tail(future_months)

    for _, row in future_forecast.iterrows():
        forecast_rows.append({
            "Date": row["ds"],
            "Cell": cell,
            "Forecast_Traffic_GB": round(row["yhat"], 1)
        })
```

STEP 5 – Build Forecast KPI Report

```
forecast_df = pd.DataFrame(forecast_rows)

forecast_df.head(10)
```

STEP 6 – Forecast Validation (Visual & Logical)

```
sample_cell = df["Cell"].iloc[0]

cell_df = df[df["Cell"] == sample_cell].rename(
    columns={"Date": "ds", "Traffic_GB": "y"}
)

model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False
)

model.fit(cell_df)

future = model.make_future_dataframe(periods=6, freq="ME")
forecast = model.predict(future)

model.plot(forecast)

# Validation plot
# Black dots -> historical traffic
# Blue line -> forecast trend
# Shaded area -> uncertainty
```

STEP 7 – Save Forecast Output as CSV

```
forecast_df.to_csv(
    "Traffic_Forecast_Next_6_Months_Per_Cell_Prophet.csv",
    index=False
)

print("Forecast KPI report saved successfully")
```

Key Takeaways for Participants

Trainer should explicitly say:

1. AI learns from **existing KPIs**
2. No RF knowledge is lost
3. One model per cell = realistic
4. Forecast output is KPI-like
5. AI supports planning decisions